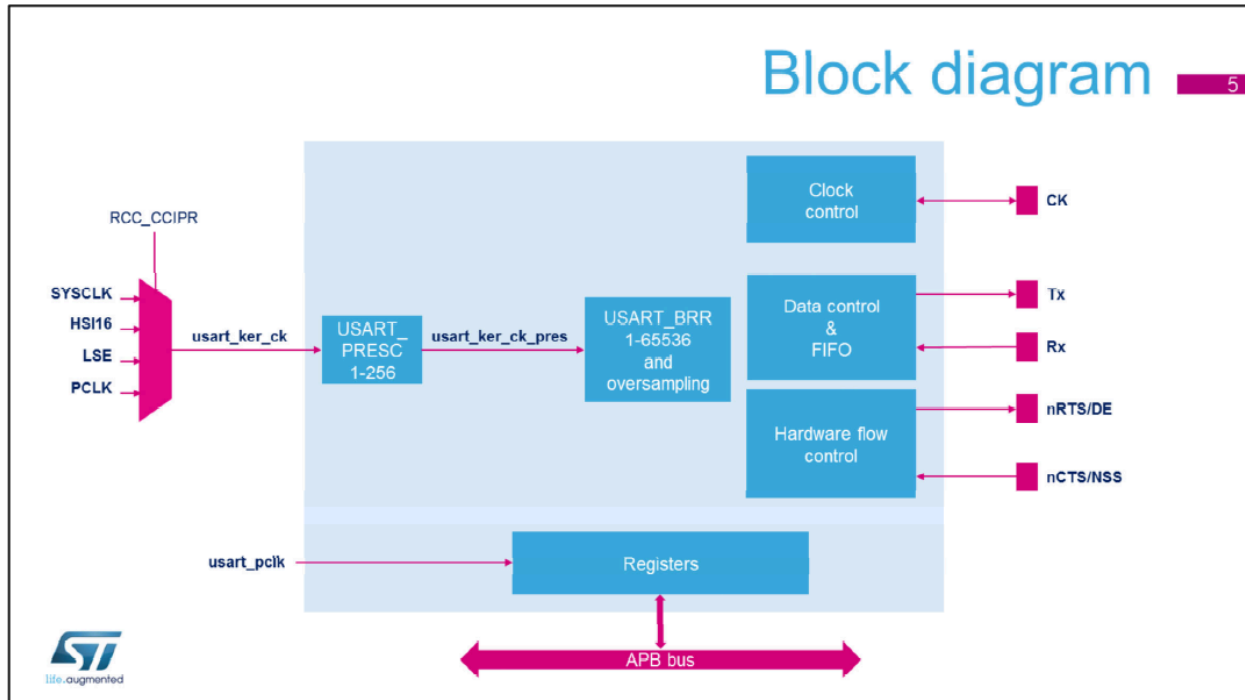


UART and USART:

Universal Synchronous/ Asynchronous Receiver/Transmitter



USART is a **serial communication interface** used to transmit and receive data between devices. It's very common in embedded systems like STM32.

*Key Functions Supported:

- **UART (Asynchronous)** – classic serial comm (e.g., PC-to-microcontroller)
- **SPI** – master/slave synchronous data
- **SmartCard** – ISO7816 protocol (used in secure cards)
- **IrDA** – infrared communication
- **LIN** – low-cost network in automotive

Major Point: Very versatile—works in many communication modes, low power friendly, with hardware FIFOs.

***USART is Configurable:**

You can **customize** these parameters:

- Data bits (7, 8, or 9)
- Parity (for error detection)
- Stop bits (0, 1, 1.5, or 2)
- Data order (MSB or LSB first)
- Baud rate (communication speed)
- Oversampling (8 or 16x – affects speed and reliability)

Major Point: You can tweak performance vs. reliability with oversampling (more on that below). Use oversampling=8 for speed, 16 for reliability

Also supports:

- **Flow control:** RTS (Request to Send), CTS (Clear to Send)
- **RS-485 driver enable (DE)**
- **Dual clock domain:** works independently of main CPU clock

***RS-485 Communication (Simple)**

RS-485 is like a **walkie-talkie** for multiple devices on the **same wire**. Only one device can **talk at a time**, others listen.

- **DE (Driver Enable)** pin is used to tell:
“I want to talk now!”
- Common in **industrial systems** where you connect **many devices on long wires**.
- **Main Point:** DE pin is **on = transmit, off = receive**. This helps prevent everyone from talking at once!

***RS-232 Communication (Simple)**

RS-232 is like **two phones** connected directly by wires.

It uses:

- **TX** – transmit
- **RX** – receive
- **CTS (Clear To Send)** – Are you ready?
- **RTS (Request To Send)** – I want to send!

RS-232 helps avoid data **collisions** (two devices talking at the same time).

Main Point: Uses extra lines (RTS/CTS) to make sure the devices talk only when both are ready.

***Serial vs. Parallel Communication:**

- **Serial Communication:** Data is sent one bit at a time over a single wire.
 - Pros: Fewer wires, cheaper, and easier to implement.
 - Cons: Slower than parallel.
- **Parallel Communication:** Data is sent multiple bits at once (e.g., 8 bits at a time over 8 wires).
 - Pros: Faster because more data is transmitted simultaneously.
 - Cons: Needs many wires, making it bulkier, more expensive, and harder to manage.

***Master/Slave Communication:**

- One device (master) controls the communication, and the other (slave) follows.
- The **master provides the clock signal** to synchronize the timing.

***Clock Usage:**

- In synchronous systems, a clock is used so both sender and receiver stay in sync.
- In asynchronous systems, timing is agreed upon ahead of time; no clock signal is shared.

***Differential Signaling:**

- Involves sending the same signal over **two wires**, but one is inverted.
- The receiver compares the difference, which cancels out any noise picked up along the way.

***Twisted Pair Wires:**

- Twisting the wires together helps reduce **electromagnetic interference (noise)**.
- Common in communication cables like Ethernet.

***Why Use 220V in Power Supplies?**

- **High voltage = low current for the same power.**
- Lower current means:
 - Less heat loss
 - Less voltage drop across wires
 - Thinner (cheaper) wires can be used

***Importance of Wire Gauge(Thickness):**

- **Thicker wires** can carry more current with less resistance.
- **Thinner wires** can overheat or cause voltage drop if current is too high.

***Why Less Current is Better:**

- Lower current helps avoid power losses due to resistance.
- More efficient for long-distance transmission.

***Asynchronous vs. Synchronous Communication:**

- **Asynchronous:**
 - No clock signal is shared.
 - Devices agree on a fixed speed beforehand (e.g., 9600 bits/sec).
 - Used in UART.
- **Synchronous:**
 - Devices share a clock signal.
 - Better for faster data transfer (e.g., SPI, I²C).

***RS-232 and RS-485:**

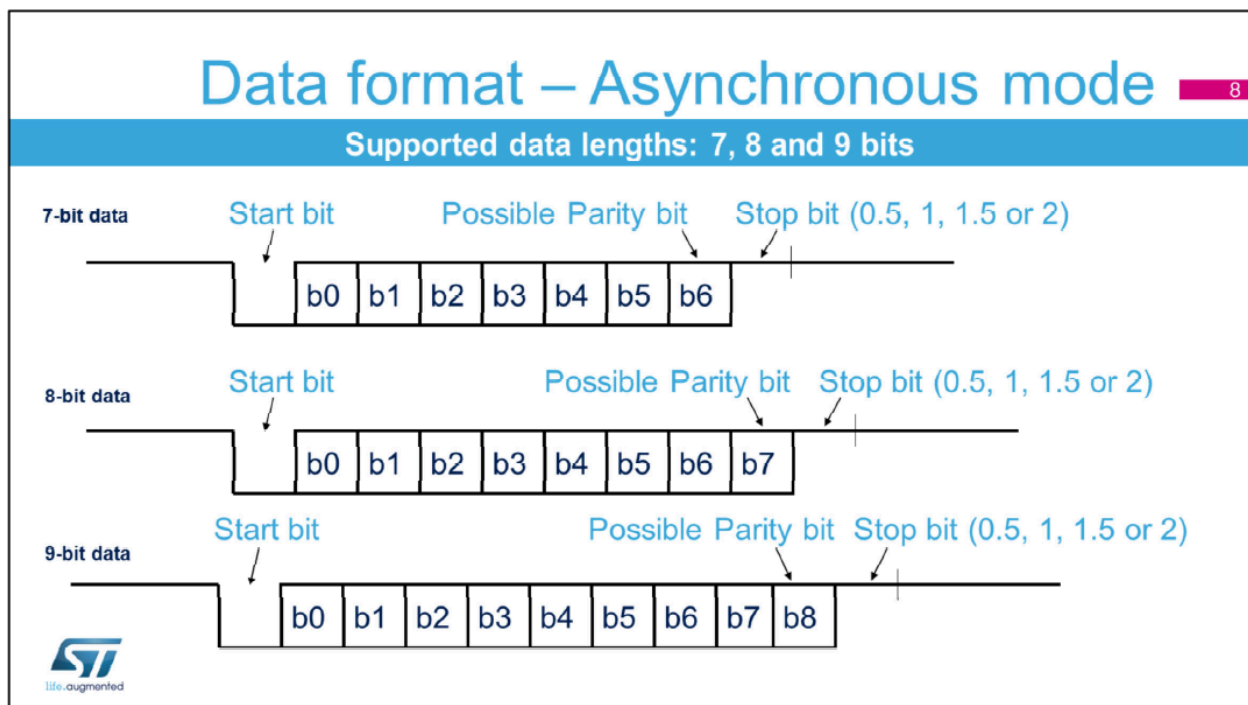
- **RS-232:** An older serial communication standard (used in PCs).
- **RS-485:** A newer standard, supports longer distances and multiple devices.

*Differential Pairing Not Used in SPI:

- SPI uses separate lines for clock, data in/out, and chip select.
- It doesn't require differential pairs since it's often used for short distances.

*Asynchronous Mode Details:

- Data is sent bit by bit, without a shared clock.
- A data packet has:
 - **Start bit:** Always LOW (0) – tells the receiver data is starting.
 - **Data bits:** Usually 8 bits.
 - **Optional Parity bit:** Used for error checking.
 - **Stop bit:** Always HIGH (1) – marks the end.



*Even and Odd Parity:

- **Even parity:** The number of 1s (including parity) is even.
- **Odd parity:** The number of 1s is odd.
- Helps catch **single-bit errors** during transmission.

*Why the UART Line Defaults HIGH (Idle State)

In **asynchronous serial communication** (like UART), there's no clock signal shared between the sender and receiver. So, the receiver must figure out **when** data starts and ends **purely by observing the data line**.

When **no data is being sent**, the **Tx (transmit)** line is **held high**. This is known as the **idle state**. It's a standard in UART.

UART Idle State = High(1)

To begin transmission:

- The sender pulls the line **low**, signaling a **start bit**.
- This drop from high to low tells the receiver, "Hey! Data is coming!"
- Then the actual data bits are sent.

*Now What Happens If the Wire is Cut or Disconnected?

If the Tx line gets **physically broken, unplugged, or disconnected**, here's what could happen:

✗ Without Default High:

- The wire might "float" — it doesn't have a definite voltage.
- It might randomly pick up noise or fluctuate.

- The receiver could **misinterpret noise as a start bit**, falsely thinking data is coming.
- This leads to **garbled or incorrect data**.

✅ **With Default HIGH:**

- The line is designed to **drift HIGH** when left unconnected or idle.
- This is often ensured using a **pull-up resistor** in the circuit.

This means:

- The line stays at a safe, predictable level (HIGH = logic 1).
- If it's disconnected, the receiver just sees idle state — no data transmission.
- It acts like a **fail-safe**: prevents **false triggering** and **random noise from being interpreted as data**.

***Pull-up Resistor (How it's Done Physically):**

- A small resistor connects the Tx line to a positive voltage (e.g., +5V).
- When nothing is driving the line (i.e., no data being sent), the resistor **pulls it up to HIGH**.
- But if the transmitter actively sends LOW (start bit), it **overrides the pull-up**.

Summarising:

- **Default HIGH** ensures the line is stable when not in use.
- Prevents **noise or disconnection** from being mistaken as real data.
- Helps the receiver stay in sync and wait calmly for valid data.

***Half Duplex Communication:**

- Same wire used to **send and receive**, but not at the same time.
- Example: walkie-talkies — one speaks while the other listens.

***Full Duplex:**

- Data can be **sent and received at the same time** using separate lines.
- Example: phones — both can talk and hear simultaneously.

***Transceiver States:**

- Transceivers (transmitter + receiver) need to manage who is sending/receiving.
- If both try to send at the same time in half-duplex, **data collision** occurs.

***Prescaler:**

- A prescaler **divides the main clock** frequency to get a lower frequency suitable for UART.
- It sets the **baud rate** for asynchronous communication.

***Baud Rate:**

- Number of **bits per second**.
- Common rates: 9600, 115200 bps.
- Higher baud rate = faster communication.

***RS-485 Communication:**

- Supports **multiple devices** on the same bus.
- Uses **differential signaling**, great for longer distances and industrial systems.

***PRBS (Pseudo Random Bit Sequence):**

- Used in **testing communication lines**.
- A known pattern of bits is sent and checked to verify the link.

***Applications (like Attendance Systems):**

- UART is used in systems like RFID-based attendance.
- Cheap, reliable, and doesn't need a complex setup.

***Rx/Tx Lines:**

- **Rx (Receive)** and **Tx (Transmit)** are the two main UART pins.
- Data from Tx of one device connects to Rx of another.